

code? The original intent of wrapping the `my_data` class in the wrapper class `my_data_wrapper` was to make sure that all users end up using the `'my_data'` class after acquiring the necessary lock, namely, `'my_data_mutex'`. This is achieved by calling the `'PerformOperation'` function of `my_data_wrapper` class with the appropriate function `'func'` to be invoked, so that the lock `'my_data_mutex'` is first acquired and then the function `'func'` is performed, and once the function `'func'` is completed, the mutex is released as well. However, it is possible to circumvent this protection mechanism, as shown above, by storing the pointer to the 'sensitive'/'protected' data in a global pointer and using the pointer to invoke a function which operates on the protected data, directly bypassing the `'my_data_wrapper'` class. This results in bypassing the lock acquire/release and hence can lead to race conditions if executed by concurrent threads. Therefore, it is important to note that passing out pointers or references to protected data and bypassing the locks associated with them can lead to race conditions and the onus for ensuring consistency is on the programmers.

What are Lock Guards?

In the above code snippet, I had called the lock and unlock functions of `std::mutex` directly. However, the recommended practice is not to call these functions directly. Why do we want to discourage developers from calling the lock/unlock functions explicitly? The reason is that the developer has to ensure that the mutex is appropriately unlocked on every code path after the operation on the protected data is completed. For instance, if an exception is thrown inside `'myfunc'` that was called from `'PerformOperation'` in the above code snippet and `control` exits through an exception path, the lock would not be released automatically. Recall the RAII idiom which stands for Resource Acquisition Is Initialisation. In order to support RAII for mutexes, C++ provides the `std::lock_guard` class template. This class `std::lock_guard` locks the mutex supplied to it as an argument on construction and releases it on destruction. This ensures that the locked mutex is unlocked on all paths of execution when the corresponding `lock_guard` object gets destructed. Now we can rewrite the `'PerformOperation'` function of `'my_data_wrapper'` class using `std::lock_guard` as shown below:

```
Class my_data_wrapper {
    Private:
        my_data data1;

        std::mutex my_data_mutex;
        template<typename Function> void
        PerformOperation(Function my_func)
```

```
{
    std::lock_guard<std::mutex> my_lock_guard(my_data_
mutex);
    my_func(data1);
}
}
```

So far we have skirted around the issues of how an application can acquire the multiple locks it needs. For instance, consider the simple example of two tables, A and B, each protected with their own mutex locks, `A_lock` and `B_lock`, respectively. So any operation on Table A will acquire the `'A_lock'`, perform all operations on it and release the `'A_lock'`. Similar conditions hold for operations on Table B. Now consider the case of an operation `'Move_from_TableA_To_TableB'` where we want to *move* an item from Table A to Table B. We will now need to hold both locks before we perform the move. So let us acquire the lock on Table A first and then the lock on Table B, perform the `'Move'` operation and release the locks once done. Now, if there is another operation `'Move_from_TableB_To_TableA'`, we need to ensure that the order in which the locks are acquired in this operation is the same as the order in which we acquire the locks in `'Move_from_TableA_To_TableB'`. Else, it can lead to a deadlock situation where one thread which is performing `'Move_from_TableA_To_TableB'` has acquired the lock for Table A and is waiting for the lock for Table B, while another thread which is performing `'Move_from_TableB_To_TableA'` has acquired the lock for Table A and is trying to acquire a lock for Table B. Hence, it is necessary for the programmer to define and follow a lock order in order to avoid deadlocks. C++ helps the programmer by providing the `std::lock` function. This function can take multiple arguments of type `std::mutex` and can lock them without the possibility of a deadlock.

```
std::mutex my_mutex1;
std::mutex my_mutex2;
Void foo()
{
    std::lock(my_mutex1, my_mutex2);
    //do work
}
```

`std::lock` provides an 'all or nothing' semantics when acquiring the locks. Either both the locks are acquired or neither are. In case an exception is hit after `std::lock`, it acquires the first mutex successfully but before acquiring the second mutex, the first mutex is automatically released. However, it is still